

Facultad de Ciencias de la Administración

Universidad Nacional de Entre Ríos

PROYECTO – LÓGICA PARA LAS CIENCIAS INFORMÁTICAS

MOTOR DE INFERENCIA Y ANÁLISIS DE REDES SOCIALES COMPLEJAS EN PROLOG

Objetivos del Proyecto

- **Modelado Lógico Avanzado:** Diseñar e implementar una base de conocimiento dinámica en Prolog a partir de fuentes de datos semiestructuradas (JSON).
- **Resolución de Entidades e Integridad:** Resolver la impedancia de datos e inconsistencias de identificadores locales mediante técnicas de normalización en memoria.
- **Inferencia sobre Grafos:** Desarrollar predicados recursivos y condicionales avanzados para extraer patrones topológicos e información implícita en redes complejas.
- **Defensa Técnica:** Demostrar dominio del paradigma lógico mediante un coloquio oral y la justificación algorítmica del código entregado.

2. Descripción General del Problema

En el análisis de redes sociales (SNA), las interacciones físicas o de habla suelen omitir interacciones implícitas críticas. Un caso de estudio clásico es el dataset de redes sociales de **Star Wars**. En los registros puros de diálogos, personajes clave como *R2-D2* o *Chewbacca* quedan aislados por no poseer líneas de texto traducidas (hablan mediante pitidos o rugidos).

Para solucionar esto, el dataset cuenta con tres capas de información por cada episodio cinematográfico:

1. **interactions (Habla Pura):** Registra exclusivamente los diálogos directos entre personajes.
2. **mentions (Menciones):** Registra qué personaje nombra a otro, capturando la red de percepción.
3. **interactions-allCharacters (Red Completa):** Una red extendida que combina diálogos e interacciones físicas, reincorporando a los personajes "silenciosos".

El desafío del proyecto consiste en construir un sistema en Prolog que ingeste estas tres capas para los 7 episodios disponibles, unifique las identidades de los personajes bajo un catálogo global y provea un motor de consultas de alta complejidad para auditar la estructura narrativa de la saga.

3. Requerimientos de Implementación

Se deberá estructurar el código fuente en tres fases claramente diferenciadas dentro de su archivo .pl:

Fase I: Ingesta Dinámica y Normalización de Datos

El sistema no debe depender de hechos estáticos escritos en el código. Se debe implementar un predicado `inicializar_sistema/0` que automatice la lectura de los 22 archivos JSON del dataset (7 de habla pura, 7 de menciones, 7 de interacciones completas y 1 consolidado general).

Se deben respetar estrictamente las siguientes firmas dinámicas para la persistencia en memoria:

- `personaje(Nombre)` Catálogo único global. No admite duplicados.
- `aparece_en(Nombre, Contexto, Tipo)` Vincula al personaje con el episodio (1..7 o `saga_completa`) y el origen de datos (`habla_pura`, `mencion`, `interaccion_completa`).
- `relacion(Tipo, Origen, Destino, Peso, Contexto)` Arcos dirigidos del grafo social. Los nodos origen y destino deben registrarse con su **nombre real** (**String/Átomo**) y no con el índice posicional volátil del JSON.

Fase II: Motor de Consultas e Inferencia

Se requiere la implementación de los siguientes predicados de análisis estructural:

1. `personaje_silencioso/2 (+Nombre, +Episodio 0 -Nombre, +Episodio)`: Detecta "*Nodos Fantasma*". Triangula personajes que participan activamente en la red extendida de un episodio pero que tienen un peso de cero (ausencia total de arcos) en la red de habla pura.
2. `triangulacion_comunicacion/4 (-PersonajeA, -PersonajeB, +Intermediario, +Episodio)`: Encuentra patrones de aislamiento relacional. Identifica pares de personajes que jamás hablaron entre sí en una película, pero que dependen estrictamente de un intermediario común para que la información fluya.
3. `nucleo_persistente_trilogia_original/2 (-Personaje1, -Personaje2)`: Realiza una intersección de contextos históricos. Devuelve las conexiones que se han mantenido activas e ininterrumpidas a lo largo de toda la trilogía clásica (Episodios IV, V y VI simultáneamente) utilizando optimización posicional para evitar respuestas espejo.

Fase III: Interfaz de Consola y Formateo Visual

Para la auditoría del sistema, se deben proveer los predicados:

`mostrar_personajes/2 (Contexto, Tipo)`: Muestra los personajes de un contexto (episodio/saga completa) para un origen de dato.

mostrar_relaciones/3. (Tipo, Contexto, PesoMinimo): Muestra las relaciones Origen, Destino, y Peso en base al origen de dato, contexto y un peso mínimo especificado.

Estos predicados deben renderizar tablas limpias en la consola de SWI-Prolog utilizando alineación rígida por ancho de columnas (sin romper las líneas verticales por la longitud del nombre del personaje) e implementar bucles basados en fallas (`fail`) para no saturar el stack de memoria durante el volcado de datos.

4. Metodología de Evaluación y Defensa

- **Entrega del Código:** Se requiere la entrega de archivos fuente `.pl` (puede ser un único archivo o separados por fases e introducidos a un archivo principal de la forma `:- consult('archivo.pl')`). Los archivos deben estar sintácticamente limpios, libres de advertencias (*warnings*) de variables no compartidas o predicados discontinuos.
- **Instancia de Defensa (Coloquio):** La aprobación del proyecto está sujeta a una defensa oral obligatoria. Durante la misma, se solicitará:
 1. Explicar el flujo de control del backtracking durante la carga dinámica de datos.
 2. Modificar en tiempo real una de las consultas de la Fase II para añadir una nueva restricción (por ejemplo, filtrar por un umbral de peso acumulado).
 3. Justificar la elección de los predicados de corte o control de flujo implementados.

5. Apéndice Técnico: Guía de Predicados Predefinidos

Para la resolución de las capas de entrada/salida y el procesamiento imperativo en memoria, los alumnos disponen de los siguientes predicados nativos de SWI-Prolog. Se detalla su comportamiento interno para guiar la implementación:

A. Mecanismos de Lectura de Archivos y Flujos (Streams)

- **open(+RutaArchivo, +Modo, -Stream):** Solicita al sistema operativo un descriptor de archivo para la ruta especificada. En este proyecto, se utiliza exclusivamente en Modo `read`. Instancia la variable `Stream` con un identificador lógico de flujo.
- **json_read_dict(+Stream, -Dict):** Predicado perteneciente a `library(http/json)`. Consume el flujo de texto con formato JSON y lo parsea en una estructura de datos nativa de Prolog denominada *Diccionario* (`Dict`). Los diccionarios permiten acceder a sus campos mediante la sintaxis de punto (ej: `Dict.nodes`).
- **close(+Stream):** Cierra el descriptor de archivo abierto. Es una buena práctica de ingeniería para liberar los recursos del sistema operativo inmediatamente después de volcar el JSON a la memoria.

B. Predicados de Control e Iteración sobre Listas

- **nth0(?Index, ?Lista, ?Elemento):** Predicado reversible de control posicional. En este proyecto se utiliza en la forma `nth0(-Index, +Lista, -Elemento)` para recorrer el arreglo de nodos del JSON. Captura de forma simultánea el elemento actual (`Nodo`) y su posición física en el arreglo (`Index`), lo cual es indispensable para el mapeo posterior de los arcos.
- **member(?Elemento, +Lista):** Comprueba si `Elemento` pertenece a la `Lista`. Al ser usado con una variable libre en el primer argumento, actúa como un generador, unificando secuencialmente cada enlace (`Link`) del arreglo de arcos mediante *backtracking*.
- **forall(:Condicion, :Accion):** Estructura de control de orden superior. Evalúa matemáticamente la expresión: $\forall x (\text{Condicion}(x) \rightarrow \text{Accion}(x))$. Forzar un fallo interno controlado después de cada ejecución de la `Accion` evita la acumulación de entornos en el stack, permitiendo iterar de manera lineal.

C. Manipulación Dinámica de la Base de Conocimiento

- **assertz(+Clausula):** Modifica la base de conocimiento en tiempo de ejecución. Inserta la `Clausula` (un hecho o una regla) al **final** del predicado dinámico correspondiente. Asegura que los datos conserven el orden de lectura original del archivo.
- **retractall(+Cabeza):** Elimina de la memoria interna todos los hechos cuyas estructuras unifiquen con el patrón especificado en `Cabeza`. Se utiliza al inicio del programa para garantizar la idempotencia de la carga y para limpiar las tablas temporales de transbordo.

D. Formateo de Salida y Gestión de Estados Libres de Backtracking

- **format(+Formato, +ListaArgumentos):** Motor de impresión formateada en consola. Procesa los caracteres de escape dentro de la cadena de control para estructurar la salida:
 - `~w`: Imprime el argumento en su formato estándar de átomo o número.
 - `~N|`: Fuerza un salto horizontal rígido hasta la columna física `N` de la consola, rellenando el espacio intermedio con caracteres vacíos. Garantiza la simetría de las tablas de datos.
 - `~n`: Salto de línea.
 - `~d`: Entero decimal.
- **arg(+Indice, +Termino, ?Valor):** Accede de forma unificada al argumento situado en la posición `Indice` dentro de un término estructurado compuesto.
- **nb_setarg(+Indice, !Termino, +Valor):** Realiza una asignación **no destructiva y no sujeta a backtracking** (*Non-Backtrackable*). Permite mutar el valor de un argumento dentro de un término compuesto. Es la herramienta utilizada para implementar contadores globales e incrementar el total de registros listados dentro de un bucle de falla sin que el retroceso de Prolog destruya el valor acumulado.